

Teste de Estresse

Sistema de Adoção de Pets (Petz) · API Node.js + Express + PostgreSQL · Guia para rodar e entender cada resultado

Do que se trata, em uma frase

Este teste prova que, quando **vários usuários tentam adotar o mesmo pet ao mesmo tempo**, o sistema deixa **apenas um** conseguir — e os outros recebem um aviso de "já foi adotado". Esse comportamento é exigido pelo requisito **RF014** do projeto.

A analogia mais simples possível

Imagine uma loja com **1 brinquedo só** na prateleira. De repente, **8 crianças** esticam a mão para pegar no mesmo segundo. Quem leva? Só pode ser **uma**. O nosso sistema é o "caixa da loja": ele precisa garantir que o brinquedo seja vendido **uma única vez**, mesmo com todo mundo pedindo junto. O teste de estresse é a gente *fingindo* ser essas 8 crianças para ver se o caixa não se atrapalha.

O que você vai encontrar neste guia

1. A pergunta central: por que um usuário "ganha"?
2. O que foi incluído no código para o teste funcionar
3. Os 4 testes que existem
4. Como executar — passo a passo (com prints)
5. Onde os resultados aparecem (3 lugares)
6. Lendo o CONSOLE — linha por linha
7. Lendo o LOG (`log.jsonl`) — cada campo explicado
8. Lendo o RELATÓRIO HTML — seção por seção
9. O que validar (checklist) + glossário + cola de comandos

Como estudar por este material

Leia na ordem. Cada seção explica primeiro *o conceito* (com analogia) e depois *onde ver na prática*. Os números e imagens deste guia são de **execuções reais** que já rodaram na máquina — então o que você vai ver no seu teste será muito parecido.

1. A pergunta central: por que um usuário "ganha"?

Quando 8 pessoas clicam "adotar" no mesmo pet ao mesmo tempo, é natural pensar que *"ganha quem tem a internet mais rápida"* ou *"quem clicou primeiro"*. **Está errado.** E provar isso é o coração deste trabalho.

Quem ganha é **quem o banco de dados (PostgreSQL) consegue atender primeiro**. Para entender, siga a viagem de um clique, em 4 etapas:

#	Etapa	O que acontece (em linguagem simples)
1	Programa que simula os usuários	Dispara os 8 pedidos "juntos". Mas o computador é como um caixa único: ele atende um de cada vez , muito rápido. Então os pedidos saem quase juntos, não exatamente juntos.
2	Viagem até o servidor	Cada pedido atravessa a rede e chega ao servidor num instante levemente diferente. O servidor anota a ordem de chegada (1º, 2º, 3º...).
3	Fila do banco	O servidor tem um número limitado de "guichês" para falar com o banco (o <i>pool de conexões</i>). Se chegam mais pedidos que guichês, eles fazem fila .
4	A trava da prateleira 🔑	Aqui a corrida é decidida. O banco "tranca" a linha do pet enquanto um pedido mexe nela. O primeiro a pegar a trava encontra o pet disponível, marca como adotado e vence . Todos os outros, quando chegam na trava, já encontram o pet adotado → recebem "já foi adotado".

 **A frase que você deve saber dizer ao professor**

"Quem ganha é o **primeiro comando UPDATE** a obter a **trava de linha no banco**. A baixa latência do vencedor é **consequência** (ele não esperou ninguém), e não a causa. Por isso, às vezes o vencedor nem foi o primeiro a chegar no servidor — e quem teve a resposta mais rápida pode ter perdido."

 **Analogia da trava**

Pense numa cabine de banheiro com **tranca**. Oito pessoas chegam quase juntas. A primeira que gira a tranca entra. As outras puxam a porta, veem "ocupado" 🚫 e desistem. Não importa quem correu mais no corredor — importa **quem girou a tranca primeiro**. No banco de dados, a "tranca" é a *trava de linha* (row lock).

2. O que foi incluído no código para o teste funcionar

Cinco peças trabalham juntas. Aqui está o que cada uma faz e por que existe.

2.1 — O endpoint de adoção "atômico" (a regra que garante 1 vencedor)

No arquivo `backend/src/controllers/animaisController.js`, a adoção é feita com **um único comando** que só muda o pet se ele *ainda* estiver disponível:

```
UPDATE animais SET status = 'I' WHERE id = ? AND status = 'D'
```

Traduzindo: "marque este pet como Indisponível, **mas só se ele ainda estiver Disponível**." O banco executa isso com a trava de linha. O primeiro recebe "1 linha alterada" (venceu); os outros recebem "0 linhas alteradas" (o `WHERE` não bateu, porque já estava Indisponível) e levam o status `409`.

Por que isso é seguro?

Porque a checagem ("está disponível?") e a alteração ("marca como adotado") acontecem na **mesma instrução**, indivisível. Não dá tempo de dois pedidos "passarem" no meio. Esse é o significado de **atômico**.

2.2 — A "caixa-preta do avião": instrumentação do servidor

Para conseguir **explicar** quem ganhou (e não só dizer "ganhou"), o servidor foi ensinado a anotar 3 informações novas em cada resposta de adoção:

Campo novo	O que registra	Para que serve
<code>ordemChegada</code>	A ordem em que o servidor recebeu o pedido (1º, 2º, 3º...)	Mostrar que "chegar primeiro" não garante a vitória
<code>recebidoEm</code>	O instante exato em que o servidor recebeu o pedido	Comparar com a hora de envio e de resposta
<code>dbMs</code>	Quanto tempo o pedido passou <i>dentro</i> do banco	Mostrar onde a corrida é realmente decidida

2.3 — Os programas que simulam os usuários (os scripts de teste)

Ficam em `backend/scripts/`. Eles "fingem" ser dezenas ou milhares de pessoas clicando ao mesmo tempo, e gravam tudo o que aconteceu. São os 4 testes da Seção 3.

2.4 — A biblioteca que gera os relatórios

O arquivo `backend/scripts/lib/relatorio.js` pega tudo que foi gravado e produz **três coisas** a cada execução: o `log.jsonl` (registro cru), o `summary.json` (resumo) e o `report.html` (relatório visual). Tudo numa pasta nova dentro de `backend/out/`.

2.5 — Dois cálculos que estavam errados e foram corrigidos

O que estava errado	O conserto
Percentil (p95/p99): com poucas amostras, o cálculo "colava" no valor máximo, ficando sem sentido.	Trocado pela interpolação linear — o mesmo método do Excel e do NumPy. Agora os percentis ficam corretos e bem distribuídos.
Throughput (req/s): dividia pelo tempo <i>configurado</i> (ex.: 6s), mas a navegação acabava em ~1s. O número saía até 5x menor que o real.	Agora divide pela janela real (do 1º envio à última resposta). O número passa a refletir a vazão de verdade.

Bônus encontrado

Havia um defeito em que as estatísticas de *adoção* eram calculadas em cima dos dados de *navegação* por engano — por isso os dois blocos saíam idênticos no resumo. Também foi corrigido.

3. Os 4 testes que existem

Do mais simples (didático) ao mais pesado (carga real). Todos geram relatório HTML.

Teste	Comando	O que faz
1. Simples	<code>npm run stress</code>	N usuários tentam adotar 1 pet ao mesmo tempo. É o mais fácil de entender — comece por ele.
2. Simples automático	<code>npm run stress:full</code>	Igual ao teste 1, mas sobe o servidor sozinho antes de rodar e o desliga no fim. Bom quando você não quer abrir 2 terminais.
3. Cenário reduzido	<code>npm run stress:cenario</code>	Mais realista: 100 pessoas navegando no site + 5 corridas de adoção em paralelo (100 adotantes em 5 pets). ~550 pedidos.
4. Cenário cheio	<code>npm run stress:cenario:cheio</code>	Carga pesada: 1000 navegando + 1000 adotando (em 5 ondas). ~6000 pedidos. Mostra que o sistema aguenta volume.

Qual usar na apresentação?

Mostre o **Teste 1** para explicar o conceito (poucas linhas, fácil de apontar quem ganhou) e depois o **Teste 4** para impressionar com a escala (1000 disputando, ainda assim 1 só vence). O Teste 3 é um ótimo meio-termo.

O que muda no Teste 1 (variáveis)

Variável	Padrão	Para que serve
<code>ANIMAL_ID</code>	1	Qual pet será disputado. Use um que ainda esteja disponível.
<code>NUM_USUARIOS</code>	3	Quantas pessoas vão disputar. Ex.: 8.

4. Como executar — passo a passo

Antes de tudo (uma vez só)

1. Ter o **PostgreSQL** rodando, com o banco `petz` criado e populado (arquivos `create-table.sql` e `insert.sql`).
2. Instalar as dependências: entre na pasta `backend` e rode `npm install`.
3. Conferir a senha do banco em `backend/src/config/db.js` (se precisar).

🤖 Por que 2 terminais?

Pense num restaurante: um **garçom fixo** (o servidor) precisa ficar de pé o tempo todo esperando pedidos. Você não pode usar a boca dele para mandar os pedidos também. Então você usa **outra pessoa** (o segundo terminal) para fazer os pedidos. Por isso: um terminal segura o servidor, o outro dispara os testes.

Passo 1 — Abrir o terminal do servidor (deixe ligado)

No VS Code, menu **Terminal** → **New Terminal**. Digite:

```
cd backend
PG_POOL_MAX=50 npm start
```

Espre aparecer `Servidor rodando em http://localhost:3001`. **Não feche esse terminal e não aperte Ctrl+C**. Ele é o garçom — precisa ficar de pé.

Por que `PG_POOL_MAX=50` ?

Aumenta os "guichês" do banco de 10 para 50. Para os testes leves não faz diferença; para o Teste 4 (cheio) ajuda a aguentar a carga. Pode usar em todos sem problema.

Passo 2 — Abrir o segundo terminal (o dos testes)

Clique no **+** ao lado do nome do terminal para abrir um novo, lado a lado. Entre na pasta:

```
cd backend
```

Passo 3 — Rodar o teste que quiser

Teste 1 (simples) — escolha um `ANIMAL_ID` disponível e quantos usuários:

```
ANIMAL_ID=96 NUM_USUARIOS=8 npm run stress
```

Teste 3 (cenário reduzido):

```
npm run stress:cenario
```

Teste 4 (cenário cheio):

```
npm run stress:cenario:cheio
```

⚠️ Atenção no Teste 1: a adoção é definitiva

Depois que um pet é adotado, ele **não volta** a ficar disponível. Para repetir o Teste 1, troque o `ANIMAL_ID` por outro que ainda esteja com status `D`. Para ver quais estão livres, rode no terminal dos testes:

```
node -e "const p=require('./src/config/db');p.query(\"SELECT id,nome FROM animais WHERE status='D' ORDER BY id LIMIT 10\").then(r=>{console.table(r.rows);p.end()})"
```

Os Testes 3 e 4 **não** têm esse problema — eles reativam os 5 pets automaticamente antes de cada corrida.

5. Onde os resultados aparecem (3 lugares)

Toda execução produz resultado em três lugares. Você vai aprender a ler os três.

Lugar	Onde fica	Para que serve
① O console	No próprio terminal, na hora	Resposta rápida: passou ou falhou, quem ganhou e por quê. (Seção 6)
② O log cru	<code>backend/out/<teste>-<data>/log.jsonl</code>	O registro detalhado de cada pedido, um por linha. (Seção 7)
③ O relatório visual	<code>backend/out/<teste>-<data>/report.html</code>	O relatório bonito, com gráficos e explicações. (Seção 8)

 **Como achar a pasta da sua execução**

No fim de cada teste, o console **imprime o caminho exato** dos arquivos. Cada execução cria uma pasta nova com a data e hora no nome (ex.: `stress-adocao-20260606-020855`), então nada é sobrescrito — dá para comparar execuções diferentes.

Como abrir o relatório HTML

No Mac, no terminal dos testes:

```
open backend/out/stress-adocao-*/report.html
```

Ou abra o VS Code, navegue até a pasta `backend/out/` , ache a pasta da sua execução, clique com o botão direito em `report.html` → **Reveal in Finder** → e dê dois cliques para abrir no navegador.

6. Lendo o CONSOLE — linha por linha

Exemplo real do **Teste 1** no pet 96 (Pipoca), com 8 usuários. Veja o que cada parte significa.

Parte A — Cabeçalho e lista de resultados

```
=== TESTE DE ESTRESSE – ADOÇÃO CONCORRENTE (RF014) ===
```

```
Animal alvo: 96 (Pipoca)
```

```
Usuários simulados: 8
```

```
Status inicial: D
```

```
Resultados:
```

```
- user-1 → 409 | "Animal já foi adotado"
- user-2 → 409 | "Animal já foi adotado"
- user-3 → 200 OK | "Adoção realizada"
- user-4 → 409 | "Animal já foi adotado"
- user-5 → 409 | "Animal já foi adotado"
- user-6 → 409 | "Animal já foi adotado"
- user-7 → 409 | "Animal já foi adotado"
- user-8 → 409 | "Animal já foi adotado"
```

- **Status inicial: D** = o pet começou Disponível (se começasse "I", ninguém poderia adotar).
- **200 OK** = adotou (venceu). Só o **user-3** conseguiu.
- **409** = chegou tarde, o pet já tinha sido adotado. É o resultado **correto** de quem perdeu.

Parte B — A tabela da corrida (a parte mais importante)

Ordenada pela **ordem de chegada no servidor**. Repare nas colunas *enviou*, *chegou* e *latência*:

chegada	usuário	enviou	chegou	resp.	status	latência	banco
1º	user-2	+56ms	+71ms	+103ms	409	47ms	28ms
2º	user-1	+36ms	+76ms	+102ms	409	66ms	23ms
3º	user-4	+56ms	+76ms	+103ms	409	47ms	23ms
4º	user-3	+56ms	+77ms	+102ms	200 OK	46ms	20ms 🏆 GANHOU
5º	user-6	+56ms	+77ms	+103ms	409	47ms	22ms
6º	user-5	+56ms	+78ms	+103ms	409	47ms	21ms
7º	user-8	+56ms	+78ms	+106ms	409	50ms	21ms
8º	user-7	+56ms	+78ms	+105ms	409	49ms	21ms

🔍 Leia estas duas linhas e tudo fica claro

- 1) O **user-3** venceu, mas chegou em 4º no servidor — ou seja, **não** foi o primeiro a chegar.
- 2) O **user-1** enviou primeiro de todos (+36ms, antes dos outros) e **mesmo assim** perdeu.

Conclusão: nem "enviar primeiro", nem "chegar primeiro" decidem. Quem decidiu foi a **trava de linha no banco**, que o user-3 pegou antes.

Parte C — A explicação automática e o veredito

🔍 POR QUE "user-3" GANHOU (e não foi "só latência"):

- 1) Chegou ao servidor na 4ª posição (de 8) — NÃO foi o primeiro a chegar, mas seu UPDATE pegou a trava de linha antes (a fila do pool reordena).
- 2) Rodou UPDATE animais SET status='I' WHERE id=96 AND status='D' e achou o pet ainda 'D' → mudou para 'I' e o banco respondeu rowCount=1 (vitória).
- 3) Os outros 7 rodaram o MESMO comando, mas o pet já era 'I'. O WHERE não casou, rowCount=0, e por isso receberam 409.
- 4) Sobre latência: ele por acaso TAMBÉM teve a menor (46ms) — mas isso é CONSEQUÊNCIA (não esperou a trava de ninguém), não a causa da vitória.

✅ **Teste PASSOU: exatamente 1 usuário conseguiu adotar.**

O **veredito verde** é o que você quer ver: exatamente 1 vencedor. Se aparecesse 2 ou mais, seria um defeito grave de concorrência (RF014 violado).

7. Lendo o LOG (log.jsonl) — cada campo explicado

Este é o registro cru. Cada **linha** é um pedido, escrito no formato JSON. "JSONL" = "um JSON por linha".

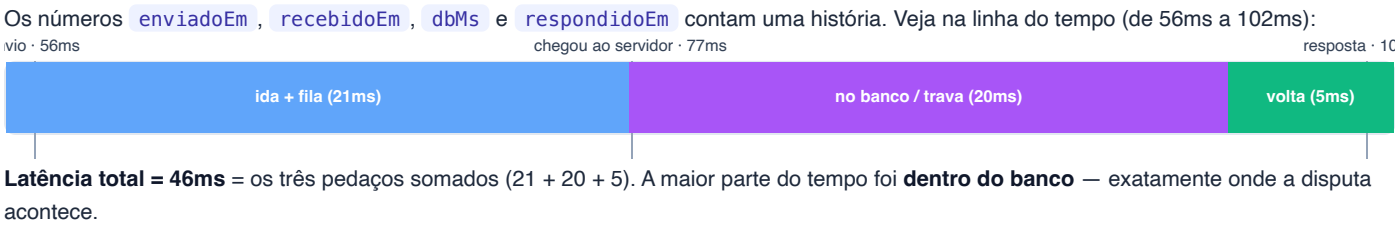
Esta é uma linha de verdade do log do pet 96 — o **vencedor** (user-3):

```
{ "tAbs": 1780722535247, "tipo": "request", "acao": "adotar", "usuarioId": "user-3",
  "petId": 96, "nome": "Pipoca", "enviadoEm": 56, "respondidoEm": 102, "latencia": 46,
  "status": 200, "motivo": "Adoção realizada", "recebidoEm": 77, "ordemChegada": 4, "dbMs": 20 }
```

O que cada campo significa

tAbs	1780722535247 — a hora exata em que a linha foi gravada, no "relógio do computador" (milissegundos desde 1970). Serve para ordenar eventos. Você raramente precisa olhar.
tipo	"request" — é um pedido HTTP (uma requisição). Todas as linhas de carga são desse tipo.
acao	"adotar" — o que o usuário tentou fazer. Pode ser <code>adotar</code> ou <code>navegar</code> (só olhando a lista).
usuarioId	"user-3" — o apelido do usuário simulado que fez este pedido.
petId	96 — o número (id) do pet disputado.
nome	"Pipoca" — o nome do pet, para facilitar a leitura.
enviadoEm	56 — quantos milissegundos depois do início do teste o cliente enviou este pedido.
recebidoEm	77 — o instante em que o servidor recebeu o pedido. Está entre o envio (56) e a resposta (102), como tem que ser.
respondidoEm	102 — o instante em que a resposta voltou para o cliente.
latencia	46 — o tempo total de ida e volta. É exatamente <code>respondidoEm - enviadoEm</code> = 102 - 56 = 46ms .
dbMs	20 — de toda a viagem, quanto tempo o pedido passou dentro do banco (fila + trava + o UPDATE). É onde a corrida é decidida.
ordemChegada	4 — este foi o 4º pedido a chegar no servidor. (E mesmo assim venceu! Prova de que chegar primeiro não é tudo.)
status	200 — deu certo, adotou. Se fosse <code>409</code> , teria perdido a corrida.
motivo	"Adoção realizada" — a mensagem do servidor. Nos perdedores aparece "Animal já foi adotado".

A viagem deste pedido, desenhada



Como é uma linha de quem PERDEU (status 409)

```
{ "acao": "adotar", "usuarioId": "ad-pet29-u46", "petId": 29, "nome": "Frida",
  "enviadoEm": 2053, "respondidoEm": 2175, "latencia": 122, "status": 409,
  "motivo": "Animal já foi adotado", "recebidoEm": 2117, "ordemChegada": 46, "dbMs": 30 }
```

Igualzinha, mas com `status: 409` e `motivo: "Animal já foi adotado"`. Este chegou em 46º — quando seu `UPDATE` rodou, o pet já estava adotado.

Como é uma linha de NAVEGAÇÃO (só olhando a lista)


```
{"acao":"navegar","usuarioId":"nav-u1","enviadoEm":0,"respondidoEm":53,"latencia":53,"status":200,"url":"http://localhost:3001/api/animais?localizacao=taguatinga","motivo":""}
```

A navegação é a "multidão de fundo" só consultando a lista de pets. Não disputa nada, então não tem `ordemChegada` nem `dbMs` — só tem a `url` que foi consultada.

8. Lendo o RELATÓRIO HTML — seção por seção

O `report.html` é o mesmo conteúdo do log, só que bonito e explicado. Abaixo, o relatório real do teste do pet 96.

Teste de adoção concorrente (RF014)

8 usuários tentando adotar o animal 96 (Pipoca) ao mesmo tempo. · 06/06/2026, 02:08:55

O que este teste verifica (RF014): quando vários usuários tentam adotar o **mesmo** pet ao mesmo tempo, o sistema deve deixar **exatamente um** vencer. Os demais recebem **409 Conflito** ("Animal já foi adotado").

Como a disputa é resolvida: a adoção roda `UPDATE animais SET status='I' WHERE id=? AND status='D'`. O Postgres dá uma **trava de linha** nessa linha: o primeiro `UPDATE` a pegar a trava muda `D→I` e vence; os outros, ao rodar o mesmo comando, já encontram `status='I'`, o `WHERE` não casa e voltam com **409**. **É o banco que serializa a corrida — não o JavaScript.**

✅ Teste PASSOU: exatamente 1 usuário conseguiu adotar.

RESUMO

ANIMAL ALVO 96 — Pipoca	USUÁRIOS 8	SUCESSOS 1 esperado: 1	CONFLITOS (409) 7	ERROS 0
TEMPO TOTAL 70ms	LATÊNCIA P50 47ms	LATÊNCIA P95 60ms	STATUS FINAL I	VENCEDOR user-3

🏆 QUEM GANHOU E POR QUÊ

Esta é a parte central: não basta dizer que alguém "chegou primeiro". Aqui mostramos, com os números reais da corrida, **por que** aquele usuário venceu — e por que a latência sozinha não explica.

Pet 96 — Pipoca 🏆 Vencedor: user-3 8 tentativas · 1 ganhou · 7 levaram 409

Por que user-3 ganhou? Chegou ao servidor na **4ª posição** (de 8) — *não* foi o primeiríssimo a chegar, mas seu `UPDATE` foi o primeiro a conquistar a trava de linha. A fila do pool de conexões pode reordenar quem encosta no banco primeiro. Neste caso o vencedor *também* teve a menor latência (46ms) — mas isso é consequência, não causa: ele não esperou a trava de ninguém, então respondeu rápido.

CHEGADA	USUÁRIO	CHEGOU EM	STATUS	LATÊNCIA (CLIENTE)	TEMPO NO BANCO
1º	user-2	+71ms	409	47ms	28ms
2º	user-1	+76ms	409	66ms	23ms
3º	user-4	+76ms	409	47ms	23ms
4º	user-3	+77ms	200 🏆	46ms	20ms
5º	user-6	+77ms	409	47ms	22ms
6º	user-5	+78ms	409	47ms	21ms
7º	user-8	+78ms	409	50ms	21ms
8º	user-7	+78ms	409	49ms	21ms

Ordenado pela **ordem de chegada no servidor**. *Chegou em* = instante (ms desde o disparo) em que o servidor recebeu o pedido. *Tempo no banco* = fila do pool + espera da trava + execução do `UPDATE`.

O que cada parte do relatório diz

Seção	O que mostra / o que olhar
Faixa azul (topo)	Explica o teste (RF014) e como a disputa é resolvida pela trava de linha. É o "resumo da teoria".
Veredito verde	O resultado final. Verde = passou (1 vencedor por pet). Vermelho = falhou. É a primeira coisa a olhar.
Resumo (cards)	Os números principais: vencedor, sucessos, conflitos (409), erros, throughput, latências. Passe o mouse em cada card para ver a explicação.
🌟 Quem ganhou e por quê	A estrela do relatório: para cada pet, explica com os números reais por que aquele usuário venceu, com a mini-tabela da corrida (o vencedor fica destacado em verde).
Timeline da corrida	Cada tentativa vira uma barrinha colorida no tempo (verde = ganhou, amarelo = perdeu). Dá para ver todo mundo disputando junto.
Throughput (vazão)	Quantos pedidos por segundo, medidos na janela real . Tem uma nota explicando por que a janela é menor que o tempo configurado.
Distribuição de latência	Um histograma: quantos pedidos foram rápidos, quantos foram lentos. Mais uma nota explicando os percentis.
Glossário	Cada termo (latência, percentil, throughput, status) explicado em uma linha.
Eventos	A tabela com todos os pedidos, filtrável por pet, status e ação. É o log em forma de planilha.

E sob carga pesada? O relatório do Teste 4 (cheio)

Teste de cenário — perfil cheio

1000 usuários navegando + 5 ondas de adoção paralelas (1000 adotantes) · 06/06/2026, 01:59:55

O que este teste verifica (RF014): quando vários usuários tentam adotar o **mesmo** pet ao mesmo tempo, o sistema deve deixar **exatamente um** vencer. Os demais recebem **409 Conflito** ("Animal já foi adotado").

Como a disputa é resolvida: a adoção roda `UPDATE animais SET status='I' WHERE id=? AND status='D'`. O Postgres dá uma **trava de linha** nessa linha: o primeiro `UPDATE` a pegar a trava muda `D→I` e vence; os outros, ao rodar o mesmo comando, já encontram `status='I'`, o `WHERE` não casa e voltam com **409**. **É o banco que serializa a corrida — não o JavaScript.**

✅ RF014 atendido: exatamente 5 sucessos (um por pet). Concorrência sob controle.

RESUMO

PERFIL cheio	USUÁRIOS NAVEGANDO 1000	USUÁRIOS ADOTANDO 1000	ADOÇÃO SUCESSO 5 esperado: 5	ADOÇÃO 409 995
ADOÇÃO ERROS 0	ADOÇÃO THROUGHPUT 5917.2 req/s 0.17s de janela	NAVEGAÇÃO TOTAL 4953	NAV THROUGHPUT 3254.3 req/s 1.52s de janela	NAV P50 2ms
NAV P95 275ms	NAV P99 362ms	PG POOL MAX 50		

🏆 QUEM GANHOU E POR QUÊ

Esta é a parte central: não basta dizer que alguém "chegou primeiro". Aqui mostramos, com os números reais da corrida, **por que** aquele usuário venceu — e por que a latência sozinha não explica.

Pet 29 — Frida 🏆 Vencedor: ad-pet29-u4 200 tentativas · 1 ganhou · 199 levaram 409

Por que ad-pet29-u4 ganhou? Chegou ao servidor na **4ª posição** (de 200) — *não* foi o primeiríssimo a chegar, mas seu `UPDATE` foi o primeiro a conquistar a trava de linha. A fila do pool de conexões pode reordenar quem encosta no banco primeiro. Neste caso o vencedor *também* teve a menor latência (100ms) — mas isso é consequência, não causa: ele não esperou a trava de ninguém, então respondeu rápido.

CHEGADA	USUÁRIO	CHEGOU EM	STATUS	LATÊNCIA (CLIENTE)	TEMPO NO BANCO
1º	ad-pet29-u1	+2114ms	409	132ms	34ms
2º	ad-pet29-u2	+2115ms	409	127ms	32ms
3º	ad-pet29-u3	+2115ms	409	127ms	33ms
4º	ad-pet29-u4	+2115ms	200 🏆	100ms	32ms
5º	ad-pet29-u5	+2115ms	409	126ms	32ms
6º	ad-pet29-u6	+2115ms	409	126ms	33ms
7º	ad-pet29-u7	+2115ms	409	126ms	33ms
8º	ad-pet29-u8	+2115ms	409	126ms	33ms

Teste 4 (cheio): 1000 navegando + 1000 adotando. Mesmo com 200 pessoas disputando o pet 29, o vencedor (ad-pet29-u4) chegou em **4º** — e ainda assim só houve 1 vencedor por pet. RF014 atendido sob carga.

 Detalhe poderoso para a apresentação

No cenário cheio, o pet 29 teve **200 disputantes** e o vencedor foi o **4º** a chegar no servidor. Isso mostra, com volume real, que a vitória não é "ordem de chegada" — é a trava do banco. Mesmo com 1000 pessoas adotando, o resultado foi **exatamente 5 vencedores** (1 por pet), 0 erros.

9. O que validar, glossário e cola de comandos

✅ Checklist — como saber que "passou"

Verifique	Resultado esperado
Quantidade de sucessos (200)	Exatamente 1 por pet. No Teste 1 = 1. No cenário = 5.
Quantidade de erros (5xx ou rede)	0
Status final do pet no banco	1 (foi adotado)
Os perdedores receberam	409 "Animal já foi adotado" (e não erro)
O veredito no console e no HTML	verde "Teste PASSOU" / "RF014 atendido"

▶ Quando é defeito?

Se aparecerem **2 ou mais sucessos no mesmo pet**, o sistema deixou o mesmo pet ser adotado duas vezes — isso é uma *race condition* e viola o RF014. O veredito ficaria **vermelho**.

📖 Glossário rápido

Termo	Em linguagem simples
Concorrência	Várias coisas acontecendo ao mesmo tempo, disputando o mesmo recurso.
RF014	O requisito do projeto que exige controlar essa disputa (1 vencedor por pet).
Trava de linha (row lock)	O "cadeado" que o banco coloca na linha do pet para um pedido mexer de cada vez.
Atômico	Uma operação indivisível: ou acontece inteira, ou não acontece. Ninguém "passa no meio".
Latência	O tempo de ida e volta de um pedido. Quanto menor, mais rápido respondeu.
Throughput (vazão)	Quantos pedidos por segundo o sistema processou.
Percentil (p95)	"95% dos pedidos foram mais rápidos que X." Mostra os piores casos, não só a média.
Pool de conexões	O número de "guichês" que o servidor tem para falar com o banco ao mesmo tempo.
Status 200 / 409	200 = deu certo (venceu). 409 = conflito (chegou tarde, perdeu).

📋 Cola de comandos (resumo)

```
# Terminal 1 – servidor (deixe ligado)
cd backend
PG_POOL_MAX=50 npm start

# Terminal 2 – testes
cd backend
ANIMAL_ID=96 NUM_USUARIOS=8 npm run stress # Teste 1 (simples)
npm run stress:full                        # Teste 2 (sobe o servidor sozinho)
npm run stress:cenario                     # Teste 3 (reduzido)
npm run stress:cenario:cheio               # Teste 4 (cheio)

# Abrir o relatório (Mac)
open backend/out/stress-adocao-*/report.html
```

🗣️ Frase de fechamento para a apresentação

"O teste de estresse simula muitos usuários adotando o mesmo pet ao mesmo tempo. Provamos, com os números reais do log e do relatório, que **só um vence** — e que o vencedor é decidido pela **trava de linha do PostgreSQL**, não pela velocidade da rede. Por isso o sistema atende ao RF014 mesmo sob carga de 1000 disputantes."